

Data of Week1

USDA – United States Department of Agriculture - rural development single family section 502 direct active borrowers by county.

Section 502 guarantees providing the loan for low- and moderate-income households to have a safe and affordable place to live in rural areas. Eligible households could purchase, build, renovate or relocate to a suitable rural area with 100% financing. There is no payment required. There is a 90% loan that does not guarantee lenders to reduce the risk of extending 100% loan to eligible buyers.¹

The attached data includes the eligible loan takers' information with attributes: number of borrowers, income levels, race, ethnicity, marital status, number of children in household, and average household size.²

¹ <https://www.rd.usda.gov/programs-services>

² <https://catalog.data.gov/dataset/usda-rural-development-single-family-section-502-direct-active-borrowers-by-county>

1.5 Python Tutorial 1

Install Anaconda

<https://docs.anaconda.com/anaconda/install/index.html>

<https://www.youtube.com/watch?v=5mDYijMfSzs>

Operations:

```
a = 2+3+5
b = 66-3-(-4)
c = 32*3
d = 2**3
e = 43/3
f = 32//3
g = 44%3
i = 44 % 3

print(e)
print(f)
print(g)
14.333333333333334
10
2
```

For Loop:

```
for x in range(0, 30, 6):
    print (x)

0
6
12
18
24
```

=====

Function:

```
def myfirstfunction(x):
    y=x**3+3*x+20
    print(y)

myfirstfunction(20)

8080
```

Statistics:

```
import math
print(math.exp(2))
print(math.log(2))
print(math.sqrt(10))
7.38905609893065
0.6931471805599453
3.1622776601683795
```

=====

Random Numbers:

```
from random import randrange
for x in range(0,3):
    print(randrange(10))

2
3
2
```

Data of Week2

USDA – United States Department of Agriculture - rural development single family section 502 direct active borrowers by county.

Section 502 guarantees providing the loan for low- and moderate-income households to have a safe and affordable place to live in rural areas. Eligible households could purchase, build, renovate or relocate to a suitable rural area with 100% financing. There is no payment required. There is a 90% loan that does not guarantee lenders to reduce the risk of extending 100% loan to eligible buyers.³

The attached data includes the eligible loan takers' information with attributes: number of borrowers, income levels, race, ethnicity, marital status, number of children in household, and average household size.⁴

³ <https://www.rd.usda.gov/programs-services>

⁴ <https://catalog.data.gov/dataset/usda-rural-development-single-family-section-502-direct-active-borrowers-by-county>

2.5 Python Tutorial 2

Read IRIS data

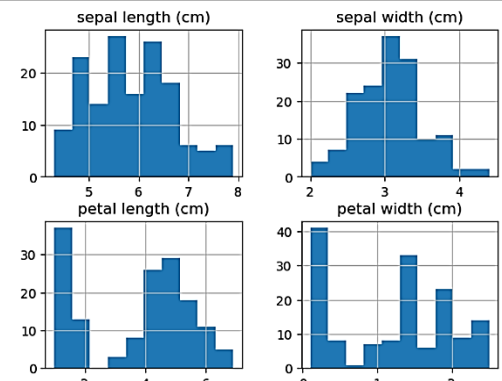
```
from sklearn.datasets import load_iris
import pandas as pd
data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Visualization

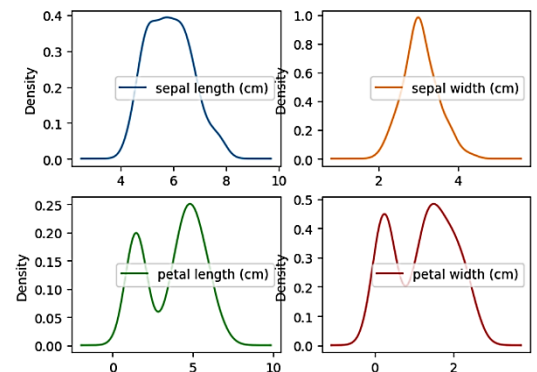
histogram

```
import matplotlib.pyplot as plt
df.hist()
plt.show()
```



Density Plot

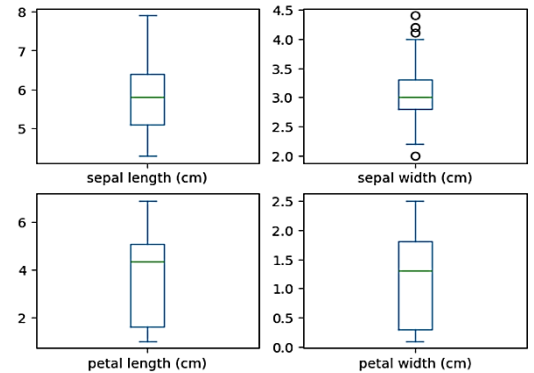
```
df.plot(kind='density',subplots=True, layout=(2,2),sharex=False)
plt.show()
```



Box Plot

```
df.plot(kind='box',subplots=True, layout=(2,2),
        sharex=False,sharey=False)

plt.show()
```



Heat Map

```
import numpy as np

names=data.feature_names

correlations = df.corr()

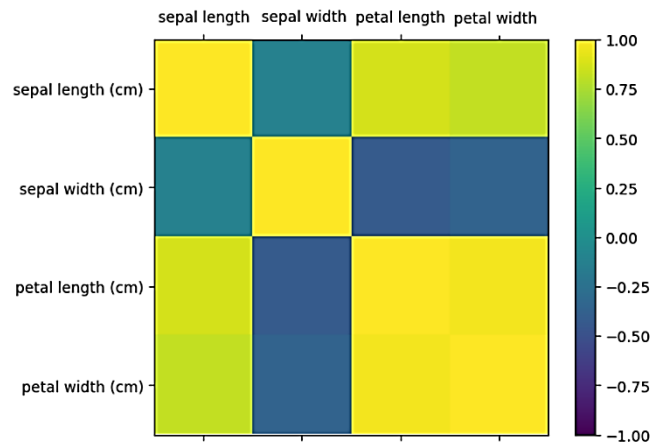
fig = plt.figure() # open the frame
ax = fig.add_subplot(111) # split figure to small parts
cax = ax.matshow(correlations, vmin=-1, vmax=1)

fig.colorbar(cax) # make the colorbar

ticks = np.arange(0,4,1) # assign numbers to ticks

ax.set_xticks(ticks) # x ticks from 0 to 3
ax.set_yticks(ticks) # y ticks from 0 to 3
ax.set_xticklabels(names) # x ticks names
ax.set_yticklabels(names) # y ticks names

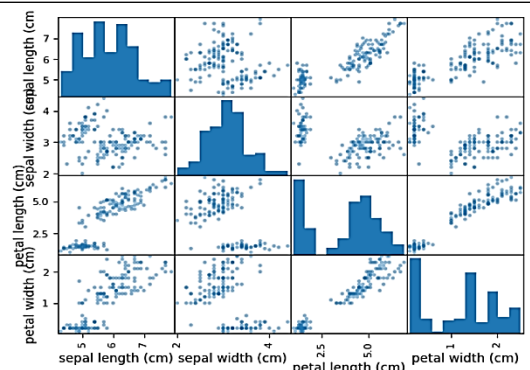
plt.show()
```



Scatter Plot

```
pd.plotting.scatter_matrix(df)

plt.show()
```



Read csv file

Upload the file in Jupyter home page.

```
import pandas
```

```
data = pandas.read_csv('Data-Week2.csv')
```

```
data.head()
```

	State-County FIPS	Total Active Borrowers	Very Low Income	Low Income	Married	American Indian/Alaskan Native	Asian	Black/African American	Hawaiin Pacific Islander	White	Hispanic	Average Number of Children	Average Household Size
0	1001.0	44	12	32	15	0.0	0.0	21.0	0.0	19	0	1.1	2.6
1	1003.0	368	157	210	82	3.0	0.0	146.0	1.0	202	8	1.1	2.2
2	1005.0	124	34	91	23	0.0	0.0	108.0	0.0	12	0	1.0	2.0
3	1007.0	20	6	14	7	0.0	0.0	11.0	0.0	9	0	0.9	2.0
4	1009.0	85	36	49	37	0.0	0.0	8.0	0.0	75	4	0.7	2.0

Bar Chart

```
import matplotlib.pyplot as plt
```

```
x = data["County"]
```

```
y = data["Married"]
```

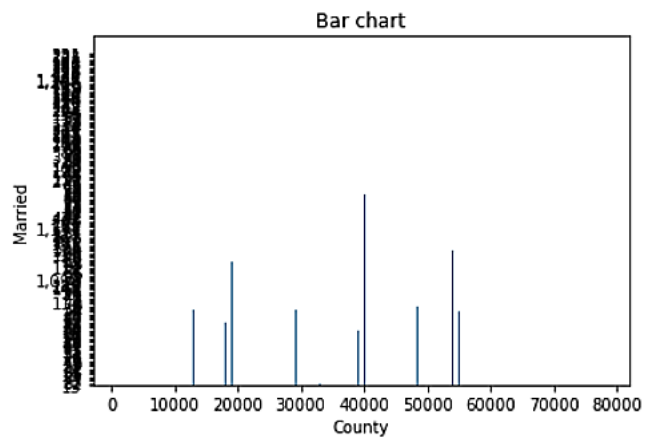
```
plt.bar(x,y)
```

```
plt.xlabel("County")
```

```
plt.ylabel("Married")
```

```
plt.title("Bar chart")
```

```
plt.show()
```



Data of Week3

Source: <https://archive.ics.uci.edu/ml/datasets/forest+fires>

This dataset is publicly available for research. The details are described in [Cortez and Morais, 2007].

Please include this citation if you plan to use this database:

P. Cortez and A. Morais. A Data Mining Approach to Predict Forest Fires using Meteorological Data.

In J. Neves, M. F. Santos and J. Machado Eds., New Trends in Artificial Intelligence, Proceedings of the 13th EPIA 2007 - Portuguese Conference on Artificial Intelligence, December, Guimaraes, Portugal, pp. 512-523, 2007. APPIA, ISBN-13 978-989-95618-0-9.

Available at: <http://www.dsi.uminho.pt/~pcortez/fires.pdf>

For more information, read [Cortez and Morais, 2007].

1. month - month of the year: 'jan' to 'dec'
2. day - day of the week: 'mon' to 'sun'
3. FFMFC - a numeric rating of the moisture content of litter and other cured fine fuels: 18.7 to 96.20
4. DMC - a numeric rating of the average moisture content of loosely compacted organic layers of moderate depth: 1.1 to 291.3
5. DC - This third moisture index reflects the moisture content of compact organic layers, 10-20 cm deep: 7.9 to 860.6
6. ISI - a numeric rating of the expected rate of fire spread. It is based on wind speed and FFMFC: 0.0 to 56.10
7. temp - temperature in Celsius degrees: 2.2 to 33.30
8. RH - a measure of how much water vapor is in the air compared to the maximum amount the air can hold at that temperature.
9. wind - removes moisture-laden air, which helps forest fuels dry out faster.

3.4 Python Tutorial 3

Read data of Week 3

import pandas as pd

data = pd.read_csv('Data-Week3.csv')

data.head(3)

	month	day	FFMC	DMC	DC	ISI	temp	RH	wind
0	1	sat	82.1	3.7	9.3	2.9	5.3	78	3.1
1	1	sun	18.7	1.1	171.4	0.0	5.2	100	0.9
2	2	mon	84.0	9.3	34.0	2.1	13.9	40	5.4

Correlation Heat Map

import numpy as np

import matplotlib.pyplot as plt

Compute correlation matrix

correlations = data.corr()

Plot

fig = plt.figure(figsize=(8, 6))

ax = fig.add_subplot(111)

cax = ax.matshow(correlations, vmin=-1, vmax=1)

fig.colorbar(cax)

ticks = np.arange(0, 9, 1)

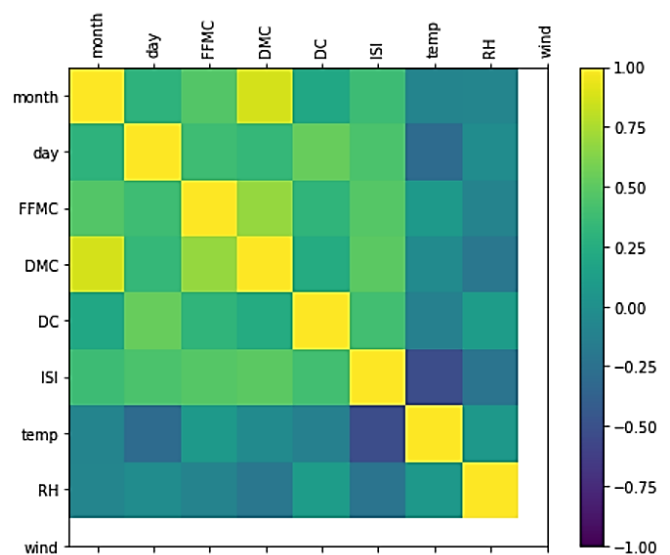
ax.set_xticks(ticks)

ax.set_yticks(ticks)

ax.set_xticklabels(data.columns, rotation=90)

ax.set_yticklabels(data.columns)

plt.show()



Interpretation:

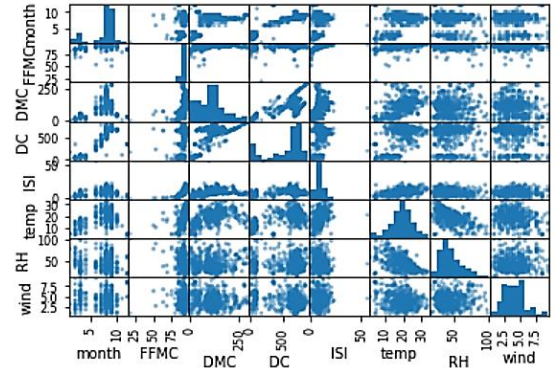
There is a high correlation between temperature and ISI (rate of fire spread which is based on wind speed and FFMC). So, we can select one of them to reduce the dimensionality.

Scatter Plot

```
import pandas as pd
pd.plotting.scatter_matrix(data)
plt.show()
```

Interpretation:

There is a high correlation between DMC & DC (moister indices). So, we can select one of them to reduce the dimensionality.



```
from numpy import set_printoptions
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2
X = data.iloc[:,2:9]
Y = data.iloc[:,0]
```

	month	day	FFMC	DMC	DC	ISI	temp	RH	wind
0	1	sat	82.1	3.7	9.3	2.9	5.3	78	3.1
1	1	sun	18.7	1.1	171.4	0.0	5.2	100	0.9
2	2	mon	84.0	9.3	34.0	2.1	13.9	40	5.4

Univariate Selection

```
test = SelectKBest(score_func=chi2, k=4)
fit = test.fit(X,Y)
set_printoptions(precision=2)
print(fit.scores_)
featured_data = fit.transform(X)
print("\nFeatured data:\n", featured_data[0:4])
```

```
[ 63.86 10355.51 54271.96 271.33 431.84 222.79 56.2 ]
```

```
Featured data:
[[ 3.7  9.3  2.9  5.3]
 [ 1.1 171.4  0.  5.2]
 [ 9.3 34.  2.1 13.9]
 [13.2 43.  5.3 12.3]]
```

Interpretation:

Selected Features: DMC, DC, temp, ISI

Univariate Selection

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
rfe = RFE(model, n_features_to_select=4)
fit = rfe.fit(X, Y)
print(fit.support_)
print(fit.ranking_)
```

```
[ True  True False False  True  True False]
[ 1  1  2  3  1  1  4]
```

Interpretation:

Selected Features: FFMC, DMC, temp, RH

```
from sklearn.ensemble import ExtraTreesClassifier
model = ExtraTreesClassifier()
model.fit(X, Y)
print(model.feature_importances_)
```

[0.1 0.24 0.37 0.09 0.09 0.06 0.05]

Interpretation:

Selected Features: FFMC, DMC, DC

Data of Week 4

This dataset contains information on all 802 Pokemon from all Seven Generations of Pokemon. The information contained in this dataset includes Base Stats, Performance against Other Types, Height, Weight, Classification, Egg Steps, Experience Points, Abilities, etc. The information was scraped from <http://serebii.net/>⁵

Selected Features:

- abilities: A stringified list of abilities that Pokemon can have
- base_total: sum of a Pokémon's base stats
- attack: The Base Attack of the Pokemon
- defense: The Base Defense of the Pokemon
- hp: The Base HP of the Pokemon
- speed: The Base Speed of the Pokemon

⁵ <https://www.kaggle.com/datasets/rounakbanik/pokemon?resource=download>

4.3 Python Tutorial 4

Read data of Week 4

```
import pandas as pd
from sklearn import linear_model
data = pd.read_csv('pokemon.csv')
data.head()
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False

Linear Regression

Predict total scores based on HP, Attack, Defense and Speed features

```
X=data[['HP','Attack','Defense','Speed']].values
```

Results:

```
y=data[['Total']].values
```

[[1.5961734 0.93753543 1.59191475 1.70552796]]

```
LR = linear_model.LinearRegression()
```

[16.48690472]

```
R = LR.fit(X,y)
```

0.888498761799349

```
print(R.coef_)
```

Interpretation:

```
print(R.intercept_)
```

Speed has the highest effect on total score.

```
print(R.score(X,y)) # score is like accuracy
```

Logistic Regression

```
import numpy as np
```

```
from sklearn import linear_model
```

```
x=np.array([3.78,2.44,2.09,0.14,1.72,1.65,4.92,4.37,4.96,4.52,3.69,5.88]).reshape(-1,1)
```

```
y=np.array([0,0,0,0,0,0,1,1,1,1,1,1])
```

```
logr=linear_model.LogisticRegression() # define the model
```

```
logr.fit(x,y) # fit the model
```

```
pred = logr.predict(np.array([1.2]).reshape(-1,1))
```

```
pred
```

```

# find the probabilities' function
def logit2prob(logr,X):
    coef=logr.coef_*X+logr.intercept_ # Linear regression coef
    odds=np.exp(coef)
    prob=odds/(1+odds) # logistic regression function e/(1+e)
    return(prob)
print(logit2prob(logr,x))

# =====
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
model = LogisticRegression(solver='liblinear',C=10,random_state=0)
model.fit(x,y)
# find the logistic probabilities
p_pred=model.predict_proba(x) # predict probabilities
y_pred=model.predict(x)      # predict the classes of each value
p_pred

# =====

```

	precision	recall	f1-score	support
0	1.00	0.83	0.91	6
1	0.86	1.00	0.92	6
accuracy			0.92	12
macro avg	0.93	0.92	0.92	12
weighted avg	0.93	0.92	0.92	12

Data of Week 5

Source: <https://archive.ics.uci.edu/ml/datasets/forest+fires>

This dataset is publicly available for research. The details are described in [Cortez and Morais, 2007].

Please include this citation if you plan to use this database:

P. Cortez and A. Morais. A Data Mining Approach to Predict Forest Fires using Meteorological Data.

In J. Neves, M. F. Santos and J. Machado Eds., New Trends in Artificial Intelligence, Proceedings of the 13th EPIA 2007 - Portuguese Conference on Artificial Intelligence, December, Guimaraes, Portugal, pp. 512-523, 2007. APPIA, ISBN-13 978-989-95618-0-9.

Available at: <http://www.dsi.uminho.pt/~pcortez/fires.pdf>

For more information, read [Cortez and Morais, 2007].

1. month - month of the year: 'jan' to 'dec'
2. day - day of the week: 'mon' to 'sun'
3. FFMFC - a numeric rating of the moisture content of litter and other cured fine fuels: 18.7 to 96.20
4. DMC - a numeric rating of the average moisture content of loosely compacted organic layers of moderate depth: 1.1 to 291.3
5. DC - This third moisture index reflects the moisture content of compact organic layers, 10-20 cm deep: 7.9 to 860.6
6. ISI - a numeric rating of the expected rate of fire spread. It is based on wind speed and FFMFC: 0.0 to 56.10
7. temp - temperature in Celsius degrees: 2.2 to 33.30
8. RH - a measure of how much water vapor is in the air compared to the maximum amount the air can hold at that temperature.
9. wind - removes moisture-laden air, which helps forest fuels dry out faster.

Read data of Week 5

```
import pandas as pd
from sklearn import linear_model
data = pd.read_csv('Data-Week5.csv')
data.head(3)
```

	month	day	FFMC	DMC	DC	ISI	temp	RH	wind
0	1	sat	82.1	3.7	9.3	2.9	5.3	78	3.1
1	1	sun	18.7	1.1	171.4	0.0	5.2	100	0.9
2	2	mon	84.0	9.3	34.0	2.1	13.9	40	5.4

Define and Split Data

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

x=data[['DMC', 'wind']]
y=data['month']

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
```

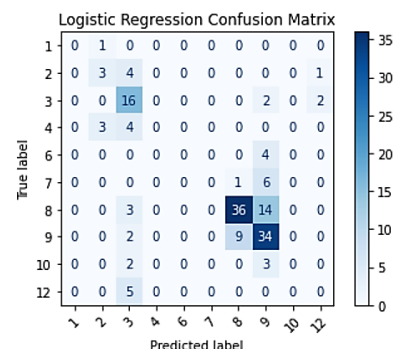
Initialize and train logistic regression

```
model = LogisticRegression(max_iter=200, multi_class='multinomial', solver='lbfgs') # good for multiclass
model.fit(x_train, y_train)

# Predict
y_pred = model.predict(x_test)
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.classes_)
disp.plot(cmap='Blues', xticks_rotation=45)
plt.title("Logistic Regression Confusion Matrix")
plt.show()
```



Accuracy

```
from sklearn.metrics import accuracy_score
```

```
acc = accuracy_score(y_test, y_pred)
```

```
print(f"Accuracy: {acc:.2f}")
```

Results:

Accuracy: 0.57

Interpretation:

Logistic Regression is not effective for prediction of this data.

Data of Week 6

Source: <https://www.kaggle.com/datasets/mahdimashayekhi/social-media-vs-productivity?resource=download>

Social Media vs Productivity — Realistic Behavioral Dataset (30,000 Users)

This dataset explores how daily digital habits — including social media usage, screen time, and notification exposure — relate to individual productivity, stress, and well-being.

The dataset contains 30,000 real-world-style records simulating behavioral patterns of people with various jobs, social habits, and lifestyle choices. The goal is to understand how different digital behaviors correlate with perceived and actual productivity.

6.3 Python Tutorial 6

Read data of Week 6

```
import pandas as pd
data = pd.read_csv('Data-Week6.csv')
data.head()
```

	age	gender	job_type	daily_social_media_time	social_platform_preference	number_of_notifications	work_hours_per_day	perceived_productivity_score
0	56	Male	Unemployed	4.180940	Facebook	61	6.753558	8.040462
1	46	Male	Health	3.249603	Twitter	59	9.169296	5.063368
2	32	Male	Finance	NaN	Twitter	57	7.910952	3.861762
3	60	Female	Unemployed	NaN	Facebook	59	6.355027	2.916332
4	25	Male	IT	NaN	Telegram	66	6.214096	8.868752

Decision Tree

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

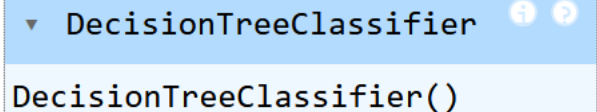
# String to Float
data['social_platform_preference'].replace(['Facebook','Instagram','Telegram','TikTok','Twitter'],[0,1,2,3,4],
inplace=True)

x=data[['social_platform_preference','sleep_hours']]
y=data['stress_level'] # the entire row of null values removed from original data

# split data to train and test
X_train, X_test, y_train, y_test = train_test_split(x,y, test_size=0.25)

# Define the model
clf=DecisionTreeClassifier()

# fit the model
clf.fit(X_train,y_train)
```



▼ DecisionTreeClassifier ⓘ ?

DecisionTreeClassifier()

Accuracy

```
from sklearn.metrics import accuracy_score
```

```
print(accuracy_score(y_true=y_train, y_pred=clf.predict(X_train)))
print(accuracy_score(y_true=y_test, y_pred=clf.predict(X_test)))
```

Results:

0.9109719058466211

0.10293280182232346

Plot Decision Tree

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
# smaller part of data for expedite the plot
x=x.iloc[0:2745,:]
y=y.iloc[0:2745]
# split data to train and test
X_train, X_test, y_train, y_test = train_test_split(x,y, test_size=0.25)
# Define the model
clf=DecisionTreeClassifier()
# fit the model
clf.fit(X_train,y_train)
print(accuracy_score(y_true=y_train, y_pred=clf.predict(X_train)))
print(accuracy_score(y_true=y_test, y_pred=clf.predict(X_test)))
plt.figure(figsize=(20,10))
plot_tree(clf,filled=True)
plt.show()
```

Results:

Accuracy:

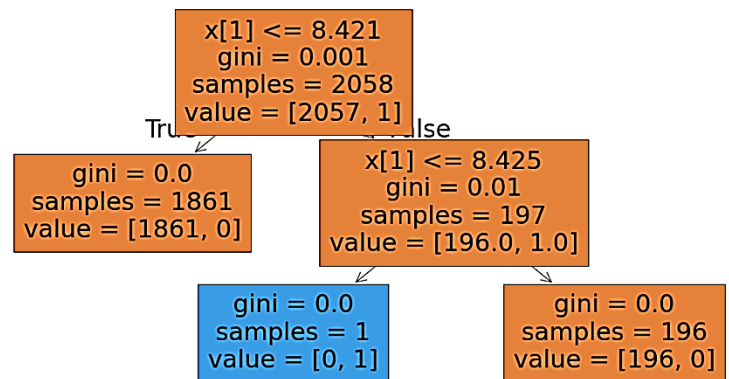
1.0

0.9985443959243085

Interpretation:

'social_platform_preference' and

'sleep_hours' effect on stress level.



Data of Week 7

<https://www.kaggle.com/datasets/shahzadahmad0402/depression-and-anxiety-data>*

Depression and anxiety data

Depression is a serious mental illness that may be identified by its distinctive symptoms, which include feelings of melancholy and emptiness, feelings of worry and disturbed sleep, as well as a general loss of initiative and interest in activities. In addition, symptoms such as the sensation of guilt or worthlessness, decreased energy, difficulties concentrating, suicidality, and psychotic symptoms may be present. The number, severity, and length of symptoms, as well as the impact on one's ability to function socially and at work, are all factors that go into determining the severity of a depression. Anxiety is a natural response to stress, and there are circumstances in which it can really be helpful. It can warn us of potential threats, assist us in preparing for them, and help us pay attention. Anxiety disorders are distinct from everyday sensations of worry or anxiousness because they include abnormally high levels of these emotions. Anxiety disorders are the most prevalent kind of mental illness, affecting roughly one in every four individuals at some time in their life. However, anxiety disorders may be treated, and there are a variety of therapies that are proven to be helpful. Most patients benefit from treatment, which enables them to enjoy normal, productive lives. Depression and anxiety are very common among students. Which affects the studies and social behavior of the students. The dataset is collected from undergraduate students at the University of Lahore. There are 787 participants who contributed to the dataset. This dataset is generated from the inspiration of the Beck Depression and Beck Anxiety inventories.

This dataset can be used as the basis for evaluating different machine learning methods and approaches such as classification of the severity of depression and anxiety. This dataset is also suitable for comparing different machine learning classification approaches.

* Records (rows) of 26, 394, 538 were removed from the original data to avoid null values of depressiveness feature.

7.3 Python Tutorial 7

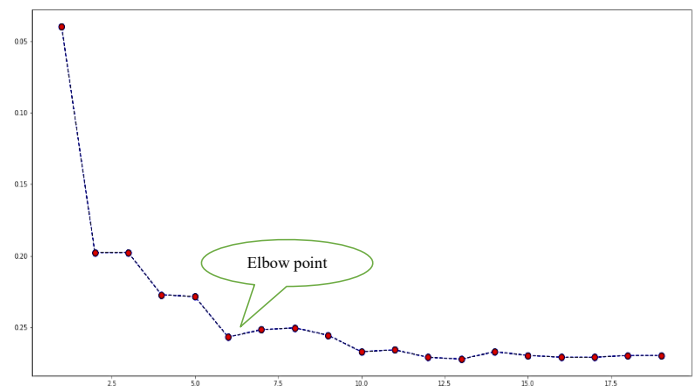
Read data of Week 7

```
import pandas as pd
data = pd.read_csv('Data-Week7.csv')
data.head()
```

	id	school_year	age	gender	bmi	who_bmi	phq_score	depression_severity	depressiveness	suicidal	depression_diagnosis
0	1	1	19	male	33.333333	Class I Obesity	9	Mild	False	False	False
1	2	1	18	male	19.841270	Normal	8	Mild	False	False	False
2	3	1	19	male	25.102391	Overweight	8	Mild	False	False	False
3	4	1	18	female	23.738662	Normal	19	Moderately severe	True	True	False
4	5	1	18	male	25.617284	Overweight	6	Mild	False	False	False

Elbow graph

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
x=data[["age","bmi"]]
y=data['depressiveness']
# Some of Square Error
SSE=[]
for k in range(1,20):
    knn=KNeighborsClassifier(n_neighbors=k)
    knn.fit(x,y)
    pred_i=knn.predict(x)
    SSE.append(np.mean(pred_i != y))
plt.figure(figsize=(20,10))
plt.plot(range(1,20),SSE,color='b',linestyle='dashed',marker='o',markerfacecolor='r',markersize=10)
# Flip graph over y axis
plt.gca().invert_yaxis()
plt.show()
```



K-NN Classifier

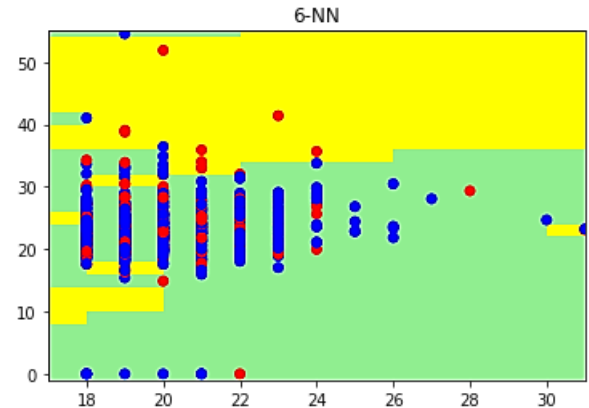
```
from matplotlib.colors import ListedColormap

n_neighbors=6
X=x.values[:,0:2]
h=2
clf=KNeighborsClassifier(n_neighbors,weights='distance')
clf.fit(X,y)

x_min, x_max=X[:,0].min()-1, X[:,0].max()+1 # x axis DMC
y_min, y_max=X[:,1].min()-1, X[:,1].max()+1 # y axis wind
xx,yy=np.meshgrid(np.arange(x_min,x_max,h),np.arange(y_min,y_max,h)) # grids
cmap_l=ListedColormap(['lightgreen','lightblue','yellow']) # predicted
cmap_d=ListedColormap(['blue','green','red']) # actual

Z= clf.predict(np.c_[xx.ravel(),yy.ravel()])
Z=Z.reshape(xx.shape)

plt.figure()
plt.pcolormesh(xx,yy,Z,cmap=cmap_l) # predicted values on back
plt.scatter(X[:,0],X[:,1],c=y,cmap=cmap_d) # actual values
plt.xlim(xx.min(),xx.max())
plt.ylim(yy.min(),yy.max())
plt.title("6-NN")
plt.show()
```



Accuracy

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2)
train_accuracy = clf.score(X_train, y_train)
test_accuracy = clf.score(X_test, y_test)
print(train_accuracy)
print(test_accuracy)
```

Results:

0.9630818619582665

0.9551282051282052

Data of Week 8

<https://www.kaggle.com/datasets/shubhambathwal/flight-price-prediction>

Flight Price Prediction

The objective of the study is to analyze the flight booking dataset obtained from “Ease My Trip” website and to conduct various statistical hypothesis tests to get meaningful information from it. The 'Linear Regression' statistical algorithm would be used to train the dataset and predict a continuous target variable. 'Easemytrip' is an internet platform for booking flight tickets, and hence a platform that potential passengers use to buy tickets. A thorough study of the data will aid in the discovery of valuable insights that will be of enormous value to passengers.

8.3 Python Tutorial 8

Read data of Week 8

```
import pandas as pd
data = pd.read_csv('Data-Week8.csv')
data.head()
```

	Unnamed: 0	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
0	203807	AirAsia	I5-517	Chennai	Morning	zero	Morning	Hyderabad	Economy	1.17	16	1105
1	203808	GO_FIRST	G8-505	Chennai	Evening	zero	Evening	Hyderabad	Economy	1.25	16	1105
2	203908	AirAsia	I5-517	Chennai	Morning	zero	Morning	Hyderabad	Economy	1.17	17	1105
3	203909	GO_FIRST	G8-505	Chennai	Evening	zero	Evening	Hyderabad	Economy	1.25	17	1105
4	204003	AirAsia	I5-517	Chennai	Morning	zero	Morning	Hyderabad	Economy	1.17	18	1105

Define X and target y

```
# Convert Destination to numerical values
```

```
data['destination_city'].replace(['Bangalore','Chennai','Delhi','Hyderabad','Kolkata','Mumbai'],[0,1,2,3,4,5],inplace=True)
```

```
X=data[['duration','price']]
```

```
Y=data['destination_city']
```

```
# Select the first 1500 rows to expedite learning
```

```
x=X.iloc[:1500,:]
```

```
y=Y.iloc[:1500]
```

Accuracy of 4 SVM kernels

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn import svm
```

```
from sklearn.metrics import accuracy_score
```

```
# define- fit - predict - find the accuracy of models
```

```
svc=svm.SVC(kernel='linear', C=1).fit(x,y) # linear kernel
```

```
svc_pred=svc.predict(x)
```

```
print('svc accuracy',accuracy_score(y,svc_pred)*100)
```

```

# continued

lin_svc=svm.LinearSVC(C=1).fit(x,y) # linear svm

lsvc_pred=lin_svc.predict(x)

print('linear svc accuracy',accuracy_score(y,lsvc_pred)*100)

# rbf kernel

rbf_svc=svm.SVC(kernel='rbf', gamma=0.7, C=1).fit(x,y)

rsvc_pred=rbf_svc.predict(x)

print('rbf accuracy',accuracy_score(y,rsvc_pred)*100)

# poly kernel, degree 3  $ax^3+bx^2+cx+d$ 

poly_svc=svm.SVC(kernel='poly', degree=3, C=1).fit(x,y)

psvc_pred=poly_svc.predict(x)

print('poly accuracy',accuracy_score(y,psvc_pred)*100)

```

Results:

svc accuracy 72.86666666666667

linear svc accuracy 58.46666666666667

rbf accuracy 100.0

poly accuracy 71.39999999999999

Interpretation:

SVM rbf is the most effective model.

```

h=10

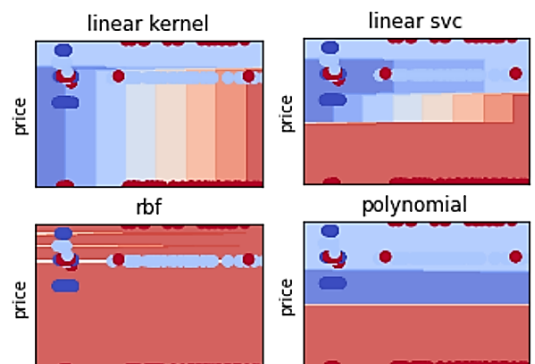
x_min, x_max = x.iloc[:,0].min()-1, x.iloc[:,0].max()+1
y_min, y_max = x.iloc[:,1].min()-1, x.iloc[:,1].max()+1
xx, yy=np.meshgrid(np.arange(x_min,x_max,h),np.arange(y_min,y_max,h))

titles=['linear kernel','linear svc','rbf','polynomial']

for i, clf in enumerate((svc, lin_svc, rbf_svc, poly_svc)):
    plt.subplot(2,2,i+1)    # create 4 subplots
    plt.subplots_adjust(wspace=0.4,hspace=0.4)
    Z=clf.predict(np.c_[xx.ravel(),yy.ravel()]) # predict destinations based on num_code and price
    Z=Z.reshape(xx.shape)
    # plot actual and predicted values
    plt.contourf(xx,yy,Z,cmap=plt.cm.coolwarm, alpha=0.8)
    plt.scatter(x.iloc[:,0],x.iloc[:,1], c=y, cmap=plt.cm.coolwarm)
    plt.ylabel('price')
    plt.xlim(xx.min(),xx.max())
    plt.ylim(yy.min(),yy.max())
    plt.xticks(())
    plt.yticks(())
    plt.title(titles[i])

plt.show()

```



Data of Week 9

<https://www.kaggle.com/datasets/subhamjain/temperature-of-all-countries-19952020>

Temperature of All Countries (1995-2020)

Contains data of various major cities of different countries in the world including daily temperature of different cities along with their country and region.

Data Preprocessing

Sort data based on Year and remove rows 2 to 412. They include invalid data.

9.3 Python Tutorial 9

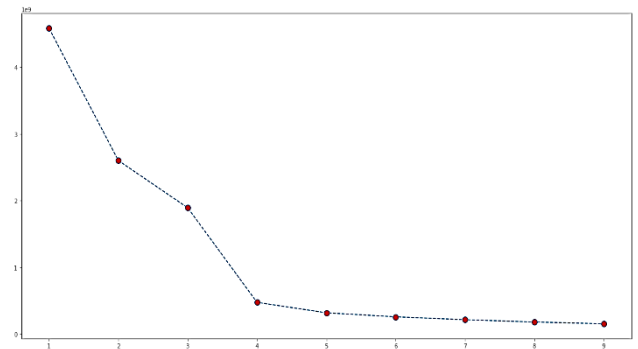
Read data of Week 9

```
import pandas as pd
data = pd.read_csv('Data-Week9.csv')
data.head()
```

	Region	Country	State	City	Month	Day	Year	AvgTemperature
0	Africa	Algeria	NaN	Algiers	1	1	1995	64.2
1	Africa	Algeria	NaN	Algiers	1	2	1995	49.4
2	Africa	Algeria	NaN	Algiers	1	3	1995	48.8
3	Africa	Algeria	NaN	Algiers	1	4	1995	46.4
4	Africa	Algeria	NaN	Algiers	1	5	1995	47.9

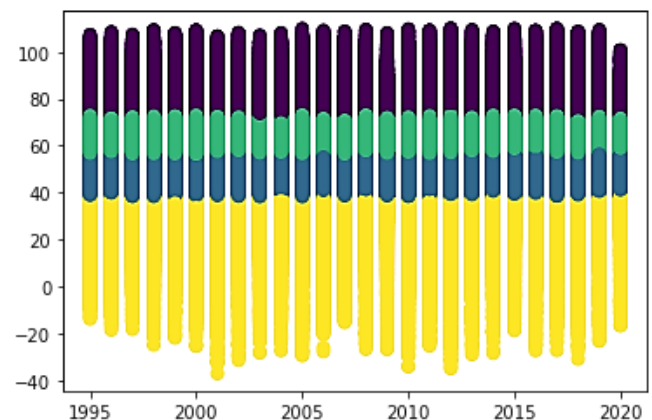
Elbow Graph

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
x=data[['Year','AvgTemperature']]
SSE=[]
for k in range(1,10):
    km=KMeans(n_clusters=k)
    km.fit(x)
    data["clusters"]=km.labels_
    SSE.append(km.inertia_) # inertia is sum of distances (actual values-predicted values)
plt.figure(figsize=(20,10))
plt.plot(range(1,10),SSE,linestyle='dashed',marker='o',markerfacecolor='r',markersize=10)
plt.show()
```



K means

```
from sklearn.cluster import KMeans
filtered_data = data[data['AvgTemperature'] > 0] # part of preprocessing data
X1= filtered_data[['Year']].values
X2= filtered_data[[' AvgTemperature']].values
km=KMeans(n_clusters=4)
km.fit(X2)
pred=km.predict(X2)
plt.scatter(X1,X2,c=pred,s=50,cmap='viridis')
centers=km.cluster_centers_
```



Data of Week 10

<https://catalog.data.gov/dataset/air-quality>

Air Quality - Metadata Updated: March 29, 2025

The data set contains information on New York City air quality surveillance data.

Air pollution is one of the most important environmental threats to urban populations and while all people are exposed, pollutant emissions, levels of exposure, and population vulnerability vary across neighborhoods. Exposures to common air pollutants have been linked to respiratory and cardiovascular diseases, cancers, and premature deaths. These indicators provide a perspective across time and NYC geographies to better characterize air quality and health in NYC. Data can also be explored online at the Environment and Health Data Portal: <http://nyc.gov/health/environmentdata>.

10.3 Python Tutorial 10

Read data of Week 10

```
import pandas as pd
```

```
data = pd.read_csv('Data-Week10.csv')
```

```
data.head()
```

	Unique ID	Indicator ID	Name	Measure	Measure Info	Geo Type Name	Geo Join ID	Geo Place Name	Time Period	Start_Date	Data Value	Message
0	336867	375	Nitrogen dioxide (NO2)	Mean	ppb	CD	407	Flushing and Whitestone (CD7)	Winter 2014-15	12/01/2014	23.97	NaN
1	336741	375	Nitrogen dioxide (NO2)	Mean	ppb	CD	107	Upper West Side (CD7)	Winter 2014-15	12/01/2014	27.42	NaN
2	550157	375	Nitrogen dioxide (NO2)	Mean	ppb	CD	414	Rockaway and Broad Channel (CD14)	Annual Average 2017	01/01/2017	12.55	NaN
3	412802	375	Nitrogen dioxide (NO2)	Mean	ppb	CD	407	Flushing and Whitestone (CD7)	Winter 2015-16	12/01/2015	22.63	NaN
4	412803	375	Nitrogen dioxide (NO2)	Mean	ppb	CD	407	Flushing and Whitestone (CD7)	Summer 2016	06/01/2016	14.00	NaN

Hierarchical Clustering

```
import numpy as np
```

```
from scipy.cluster.hierarchy import linkage, dendrogram
```

```
import matplotlib.pyplot as plt
```

```
data['Geo Type Name'].replace(['Borough','CD','Citywide','UHF34','UHF42'],[0,1,2,3,4],inplace=True)
```

```
df = data[['Geo Type Name','Data Value']]
```

```
df=df.iloc[:1000,:]
```

```
## Select 1000 rows to expedite the process
```

```
# Perform hierarchical clustering
```

```
linked = linkage(df, method='ward') # 'ward' minimizes variance within clusters
```

```
# Plot dendrogram
```

```
plt.figure(figsize=(8, 5))
```

```
dendrogram(linked,
```

```
    orientation='top',
```

```
    distance_sort='descending',
```

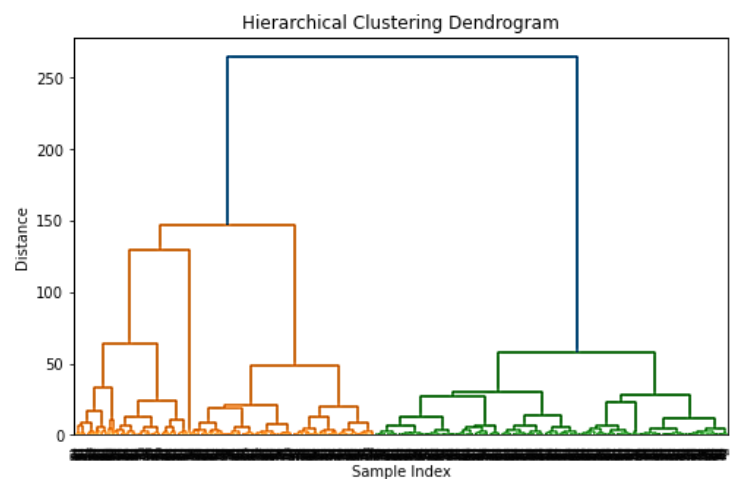
```
    show_leaf_counts=True)
```

```
plt.title('Hierarchical Clustering Dendrogram')
```

```
plt.xlabel('Sample Index')
```

```
plt.ylabel('Distance')
```

```
plt.show()
```



Data of Week 11

<https://www.kaggle.com/datasets/mathchi/diabetes-data-set>

Diabetes Dataset

This dataset is originally from the National Institute of Diabetes and Digestive and Kidney Diseases. The objective is to predict based on diagnostic measurements whether a patient has diabetes.

Content

Several constraints were placed on the selection of these instances from a larger database. All patients here are females at least 21 years old of Pima Indian heritage.

Pregnancies: Number of times pregnant

Glucose: Plasma glucose concentration 2 hours in an oral glucose tolerance test

BloodPressure: Diastolic blood pressure (mm Hg)

SkinThickness: Triceps skin fold thickness (mm)

Insulin: 2-Hour serum insulin (μ U/ml)

BMI: Body mass index ($\text{weight in kg}/(\text{height in m})^2$)

DiabetesPedigreeFunction: Diabetes pedigree function

Age: Age (years)

Outcome: Class variable (0 or 1)

Sources:

(a) Original owners: National Institute of Diabetes and Digestive and Kidney Diseases

(b) Donor of database: Vincent Sigillito (vgs@aplcn.apl.jhu.edu)

Research Center, RMI Group Leader

Applied Physics Laboratory

The Johns Hopkins University

(c) Date received: 9 May 1990

11.4 Python Tutorial 11

Read data of Week 11

```
import pandas as pd
```

```
data = pd.read_csv('Data-Week11.csv')
```

```
data.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Boosting

```
from xgboost import XGBClassifier
import xgboost as xgb
import numpy as np
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import accuracy_score
# Subset the dataset to contain only the first 100 rows
```

```
df_subset = df.head(100)
```

```
X = df_subset.drop(columns=['Outcome'])
```

```
y = df_subset['Outcome']
```

```
# Define the XGBoost classifier
```

```
clf = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
```

```
clf.fit(X, y) # Fit the model
```

```
# Perform 5-fold cross-validation
```

```
cv_scores = cross_val_score(clf, X, y, cv=5, scoring='accuracy')
```

```
print("Cross-validation scores:", cv_scores)
```

```
print("Mean cross-validation score:", np.mean(cv_scores))
```

```
# Split data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
clf.fit(X_train, y_train) # fit train set
```

```
y_pred = clf.predict(X_test) # predict the test set
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Results:

Cross-validation scores: [0.7 0.5 0.7 0.8 0.65]

Mean cross-validation score: 0.67

Accuracy: 0.7

0.06030702590942383

Load iris data

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

iris=datasets.load_iris()

x=iris.data[:, :4]

y=iris.target

X_train, X_test, y_train, y_test=train_test_split(x, y, test_size=0.3)
```

Random Forest Classifier

```
clf=RandomForestClassifier(n_estimators=50) # Define the RF model

clf.fit(X_train, y_train) # fit train set with the RF model

y_pred=clf.predict(X_test) # predict test data using RF model

# Accuracy of RF

from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

CM=confusion_matrix(y_pred, y_test)

print(CM)

AC=accuracy_score(y_pred, y_test)

print(AC)

CR=classification_report(y_pred, y_test)

print(CR)
```

Confusion Matrix

```
[[10  0  0]
 [ 0 18  0]
 [ 0  0 17]]
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	18
2	1.00	1.00	1.00	17
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

One time install

```
conda install python-graphviz # Do not install graphviz for more than one time
```

Build the Forest

```
from sklearn.tree import export_graphviz

from sklearn import tree

import graphviz

rf=RandomForestClassifier(n_estimators=2)

rf.fit(X_train,y_train)

for i in range(0,2):

    plt.figure(figsize=(20,20))

    _=tree.plot_tree(rf.estimators_[i], filled=True)
```

